

HOSPITAL MANAGEMENT SYSTEM

Python Project Documentation

Project Type	Python Console Application
Language	Python 3
Data Structures	Lists and Dictionaries
Main Modules	Hospitals, Doctors, Patients, Appointments
Interface	Command-line menu

This documentation describes the design, features, data structures, functions, menu operations, execution process, and possible future improvements of the Hospital Management System.

1. Introduction

The Hospital Management System is a basic Python-based console application designed to manage essential hospital information. It provides separate operations for hospitals, doctors, patients, and appointments. The project demonstrates fundamental Python programming concepts through a practical example.

2. Objectives

- Store hospital, doctor, patient, and appointment information.
- Allow users to add and display records.
- Provide a hospital search facility.
- Use functions to divide the program into manageable components.
- Demonstrate the use of lists and dictionaries for data management.
- Create a simple menu-driven console application.

3. Technologies and Concepts Used

Concept	Purpose in the Project
Python 3	Programming language used to develop the application.
Lists	Store multiple hospitals, doctors, patients, and appointments.
Dictionaries	Store the details of each individual record.
Functions	Separate different operations into reusable blocks.
if / elif / else	Process the user's menu selection.
for loops	Traverse stored records and display them.
input()	Accept information from the user.
len()	Check whether a list contains records.
.lower()	Perform case-insensitive hospital-name searching.

4. Data Structures

Four main lists are initialized at the beginning of the program:

List	Stores
hospitals	Hospital dictionaries
doctors	Doctor dictionaries
patients	Patient dictionaries
appointments	Appointment dictionaries

Each record is represented using a dictionary containing key-value pairs. For example, a hospital dictionary contains name, location, contact, beds, and departments.

5. Function Documentation

add_hospital()

Accepts hospital name, location, contact number, number of beds, and departments. Creates a hospital dictionary and adds it to the hospitals list.

display_hospitals()

Displays all stored hospital records. If no records exist, it displays an appropriate message.

add_doctor()

Accepts doctor ID, name, specialization, phone number, and experience, then stores the details in the doctors list.

display_doctors()

Displays the details of all stored doctors.

add_patient()

Accepts patient ID, name, age, gender, disease/problem, and phone number, then stores the patient record.

display_patients()

Displays all stored patient records.

book_appointment()

Accepts patient ID, doctor ID, appointment date, and time, then stores the appointment.

display_appointments()

Displays all stored appointment records.

search_hospital()

Searches for a hospital by name without being affected by uppercase/lowercase differences.

6. Menu Options

Option	Operation	Function Called
1	Add Hospital	add_hospital()
2	Display Hospitals	display_hospitals()
3	Search Hospital	search_hospital()
4	Add Doctor	add_doctor()
5	Display Doctors	display_doctors()
6	Add Patient	add_patient()
7	Display Patients	display_patients()
8	Book Appointment	book_appointment()
9	Display Appointments	display_appointments()
10	Exit	Ends the program

7. Program Flow

1. Initialize empty lists for hospitals, doctors, patients, and appointments.
2. Define functions for adding, displaying, searching, and booking records.
3. Display the Hospital Management System menu.
4. Accept the user's choice.
5. Call the corresponding function according to the selected option.
6. Display the requested information or confirmation message.
7. Option 10 displays the exit message and ends the current execution.

8. Sample Input and Output

Example: Adding a Hospital

User selects option 1 and enters the hospital name, location, contact number, number of beds, and departments. The program stores these values in a dictionary and appends the dictionary to the hospitals list.

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. Add Hospital
2. Display Hospitals
3. Search Hospital
4. Add Doctor
5. Display Doctors
6. Add Patient
7. Display Patients
8. Book Appointment
9. Display Appointments
10. Exit

Enter your choice: 1

--- Add Hospital Details ---
Enter hospital name: City Hospital
Enter location: Bhopal
Enter contact number: 9876543210
Enter number of beds: 100
Enter departments: Cardiology, General Medicine
Hospital added successfully!
```

9. Important Implementation Notes

- The program currently stores data only in memory. Data will be lost when the program terminates.
- The application is a basic console application and does not use a database.
- The number of beds and patient age are converted to integers using `int(input())`.
- Hospital search uses `.lower()` so that searches are case-insensitive.
- The current menu processes one selected option during one execution of the menu section. A while loop can be added later if a continuously repeating menu is required.

10. Possible Future Enhancements

- Use a while loop so the menu remains active until the user selects Exit.
- Add update and delete operations for all records.
- Store information permanently using files or a database such as SQLite.
- Validate IDs before creating appointments.
- Prevent duplicate hospital, doctor, and patient IDs.
- Add date and time validation for appointments.
- Add a login system with user roles.

- Develop a graphical user interface.

11. How to Run the Project

1. Install Python 3.
2. Open the project folder in VS Code or another Python IDE.
3. Open the Python source file.
4. Run the program using the Python interpreter.
5. Select an option from the displayed menu and provide the requested information.

12. Conclusion

The Hospital Management System demonstrates how basic Python programming concepts can be combined to create a practical record-management application. The project provides a foundation that can later be expanded with persistent storage, validation, authentication, and a graphical interface.